That's actually a really important design decision, and I'm glad you mentioned it before we finish the documents.

I would not keep every assignment forever.

It actually matches the philosophy you've established:

> Assignment Guy is for managing active assignments, not building an archive.

That makes the app feel lighter and keeps the feed relevant.

I'd propose something like this:

Assignment Lifecycle

Draft (optional)

↓

Active
(Visible to everyone)

↓

Due

↓

Submitted

↓

Expired

↓

Archived

↓

Deleted

The important distinction is between Archived and Deleted.

Archived (temporary): Hidden from the main feed and search by default, but kept for a short period (say 30–90 days) in case someone needs it or there are grading disputes.

Deleted (permanent): Removed after the retention period.

For Version 1, I'd even keep it simpler:

Active

Closed (deadline passed)

Auto-delete after a configurable period (e.g., 30 days)

That keeps the platform focused on current work.

I also wouldn't show expired assignments on the Home Feed. Instead:

Home Feed

Active assignments only.

Profile

Your recently completed assignments.

Then, after the retention period, they're removed automatically.

The only exception I'd consider is if the uploader or a moderator marks an assignment as Reference Material. For example:

A programming project template.

A lab manual.

A past question officially intended for revision.

Those could survive longer because they're no longer just "assignments."

But that definitely feels like a Version 2 or Version 3 feature.

I really like this idea because it reinforces the product identity.

Think about WhatsApp groups. After one semester, they're full of old PDFs that nobody cares about anymore.

Assignment Guy shouldn't become that.

When students open the app, they should mostly see what matters now.

Now let's continue.

Assignment Guy

Document 5

Database Design Specification

Version 1.0

---

1. Purpose

This document defines how data is stored, related, and managed within Assignment Guy.

The database should be normalized, scalable, and easy to maintain.

---

2. Design Principles

The database should:

Minimize duplication.

Maintain data integrity.

Support future growth.

Be easy to query.

Keep assignment data organized by course.

---

3. Main Entities

Version 1 includes these primary entities:

Users

Schools

Departments

Courses

UserCourses

Assignments

AssignmentAttachments

AssignmentContext

Discussions

Comments

Notifications

Contributions

Reports

EmergencyAnswerUsage

---

4. Users

Stores account information.

Fields:

user_id (UUID)

email

display_name

profile_photo

school_id

department_id

level

contribution_score

role

created_at

updated_at

---

5. Schools

Fields:

school_id

school_name

country

created_at

---

6. Departments

Fields:

department_id

school_id

department_name

---

7. Courses

Each course belongs to a department.

Fields:

course_id

course_code

course_name

department_id

level

semester

---

8. UserCourses

Many-to-many relationship.

Allows one student to join multiple courses.

Fields:

user_id

course_id

joined_at

---

9. Assignments

The heart of the database.

Fields:

assignment_id

course_id

uploader_id

title

description

due_date

status (Active / Closed / Archived)

created_at

updated_at

---

10. Assignment Attachments

Stores uploaded files.

Fields:

attachment_id

assignment_id

file_type

file_url

uploaded_by

uploaded_at

Supported:

Image

PDF

Future:

Voice

---

11. Assignment Context

Stores additional information.

Examples:

Lecturer instructions

Chapter

Topic

Notes

Submission format

Fields:

context_id

assignment_id

author_id

content

created_at

---

12. Discussions

Each assignment has one discussion thread.

Fields:

discussion_id

assignment_id

created_at

---

13. Comments

Fields:

comment_id

discussion_id

user_id

parent_comment_id (nullable)

message

created_at

edited_at

Version 1 supports only one level of replies to keep conversations readable.

---

14. Notifications

Fields:

notification_id

user_id

type

title

body

read

created_at

---

15. Contributions

Tracks community activity.

Fields:

contribution_id

user_id

type

assignment_id (nullable)

created_at

Contribution scores are calculated from these records rather than stored as independent events.

---

16. Reports

Fields:

report_id

reporter_id

target_type

target_id

reason

status

created_at

---

17. Emergency Answer Usage

Tracks usage limits.

Fields:

usage_id

user_id

assignment_id

unlocked_at

This ensures backend enforcement of any limits or eligibility rules.

End of Document 5 – Part 1

I also like your idea about assignments expiring because it keeps the app true to its purpose. I'll make sure we formally include an Assignment Lifecycle & Retention Policy in a later section so it's not just an implementation detail—it becomes part of the product's design philosophy. That way, anyone building the app understands that Assignment Guy is about helping students through current assignments, not becoming a permanent repository of old coursework.